SQL Class 02

SQL Constraints

Define rules that keep your data accurate, consistent, and reliable.

GREEN • WHITE THEME

Learning objectives

By the end of this class, you should be able to…

Understand constraints

Explain how constraints protect data quality.

Choose the right key

Use PRIMARY KEY and FOREIGN KEY to model relationships.

Set column rules

Apply NOT NULL, UNIQUE, and DEFAULT.

Build and troubleshoot

Combine constraints and diagnose common errors.

1. Introduction to SQL Constraints

A constraint is a rule enforced by the database.

Protect data accuracy and integrity

Prevent invalid or duplicate values

Keep relationships between tables valid

Catch mistakes at the database level

Defined when creating or altering a table

```
CREATE TABLE example (
    id INT PRIMARY KEY,
    email VARCHAR(120) UNIQUE,
    full_name VARCHAR(100) NOT NULL,
    status VARCHAR(20) DEFAULT 'active'
);
```

Think of constraints as guardrails for your data.

2. PRIMARY KEY

Uniquely identifies each row in a table.

Unique

No two rows can share the same key.

Not NULL

Every row must have a key value.

One primary key

A table has one PK constraint; it may contain multiple columns.

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(80) NOT NULL
);
```

Example values

101 Amina
102 Karim
103 Nabila

A duplicate ID is rejected.

3. FOREIGN KEY

Links a child table to a parent table.

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(80) NOT NULL
);
```

```
CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(80) NOT NULL,
    dept_id INT,
    FOREIGN KEY (dept_id)
      REFERENCES departments(dept_id)
);
```

Parent

departments.dept_id

Child

employees.dept_id

Rule

Referenced value must exist (or be NULL if allowed).

Tip: create the parent table first, then the child table.

4. NOT NULL

Require a value for a column.

```
CREATE TABLE courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100) NOT NULL
);
```

NOT NULL blocks NULL — but it does not block an empty string ('') in MySQL.

Valid

INSERT INTO courses VALUES (1, 'SQL');

Invalid

INSERT INTO courses (course_id) VALUES (2);

Use meaningful validation when needed.

5. UNIQUE

Prevent duplicate values in a column or column combination.

```
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    email VARCHAR(150) UNIQUE,
    username VARCHAR(50) NOT NULL UNIQUE
);
```

Allowed

Different users have different emails.

Rejected

A second row uses an existing username.

MySQL note: a UNIQUE index can allow multiple NULL values. Use NOT NULL too if a value is mandatory.

6. DEFAULT

Supply a value when an INSERT omits the column.

```
CREATE TABLE tasks (
    task_id INT PRIMARY KEY,
    title VARCHAR(120) NOT NULL,
    status VARCHAR(20) DEFAULT 'Pending'
);

INSERT INTO tasks (task_id, title)
VALUES (1, 'Prepare report');
```

Result

status = 'Pending'

Important

DEFAULT is used when the column is omitted (or DEFAULT is specified).

Inserting explicit NULL is not the same as omitting a column.

7. Combining constraints in MySQL

A realistic table uses several rules together.

```
CREATE TABLE projects (
    project_id INT AUTO_INCREMENT,
    project_name VARCHAR(120) NOT NULL,
    project_code VARCHAR(20) NOT NULL UNIQUE,
    owner_id INT NOT NULL,
    status VARCHAR(20) DEFAULT 'Pending',
    PRIMARY KEY (project_id),
    FOREIGN KEY (owner_id)
      REFERENCES employees(emp_id)
);
```

PRIMARY KEY

Unique project identifier

NOT NULL + UNIQUE

Required, non-duplicate code

FOREIGN KEY

Owner must exist in employees

Ensure employees exists first.

8. Common errors & best practices

Read the error message, then check the rule being enforced.

Duplicate primary/unique value	Choose a new unique identifier/value.
Foreign key fails	Insert parent row first; check data types and indexes.
Cannot insert NULL	Provide a value or revise the column rule.
DEFAULT not applied	Omit the column or specify DEFAULT.
Constraint name conflict	Use clear, unique names for named constraints.

Best practice: use clear names, test edge cases, and keep related columns the same type.

9. Practical exercises

Try each task before revealing the solution on the next slide.

1. Create a departments table with dept_id as PRIMARY KEY and dept_name as NOT NULL.
2. Create employees with emp_id as PRIMARY KEY and dept_id as a FOREIGN KEY.
3. Make email UNIQUE and NOT NULL.
4. Give employee_status a DEFAULT value of 'Active'.
5. Predict what happens when you insert a duplicate email or an unknown dept_id.

Exercise solutions

One possible MySQL solution.

```
CREATE TABLE departments (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(80) NOT NULL
);

CREATE TABLE employees (
    emp_id INT PRIMARY KEY,
    email VARCHAR(150) NOT NULL UNIQUE,
    dept_id INT,
    employee_status VARCHAR(20)
      NOT NULL DEFAULT 'Active',
    FOREIGN KEY (dept_id)
      REFERENCES departments(dept_id)
);
```

Duplicate email

Rejected by UNIQUE.

Unknown department

Rejected by FOREIGN KEY.

Status omitted

Defaults to 'Active'.

10. Quick review

Can you explain each rule in one sentence?

PRIMARY KEY	Unique row identifier; cannot be NULL.
FOREIGN KEY	Keeps references between tables valid.
NOT NULL	Requires a non-NULL value.
UNIQUE	Prevents duplicate values under the index's rules.
DEFAULT	Provides a value when a column is omitted.